



Maincrafts Technology

Cybersecurity & Ethical Hacking Internship

Task 2 — Lab Build Report

Build Your Personal Cybersecurity Lab

Contents

1	What this task is about
2	My setup — hardware and software used
3	How the lab is laid out (network diagram)
4	Step 1 — Installing VirtualBox
5	Step 2 — Setting up Kali Linux
6	Step 3 — Deploying the target (Juice Shop / DVWA)
7	Step 4 — Checking connectivity
8	Step 5 — Testing the tools
9	Evidence collected
10	What I learnt from this

1. What this task is about

Task 2 is about setting up my own practice lab for cybersecurity. The idea is to have a safe space to try out different attack techniques and tools without touching anything on the real internet or real systems. Everything stays inside virtual machines on my own computer.

For this task I needed to get three things working:

- A Kali Linux VM that acts as the attacker machine
- A second machine or container running a vulnerable web app (Juice Shop or DVWA) as the target
- A proper network setup so the two can talk to each other but neither can reach the outside

Skills I was supposed to build

Virtualisation	Getting comfortable running more than one VM at a time and allocating resources properly
Lab isolation	Making sure nothing I do inside the lab can accidentally affect real systems
Deploying targets	Setting up intentionally vulnerable apps — Juice Shop and DVWA — so I have something safe to practice on
Networking basics	Understanding the difference between NAT (internet access) and Host-Only (isolated) adapters
Tool validation	Confirming that Nmap, Burp Suite and Wireshark all work before I start using them seriously

2. My setup — hardware and software used

Host machine specs

RAM	16 GB (minimum needed is 8 GB — having more makes things a lot smoother)
CPU	Intel Core i5 / AMD Ryzen 5, with hardware virtualisation (VT-x / AMD-V) turned on in BIOS
Disk	Around 50 GB free space set aside for VM images, snapshots, and captured files
Host OS	Windows 11 (the same steps work on macOS or Linux as well)

Software downloaded and used

Tool	Where I got it	What it's for
VirtualBox 7.x	virtualbox.org/wiki/Downloads	Runs the VMs
Kali Linux OVA	kali.org/get-kali/#kali-virtual-machines	Prebuilt attacker VM image
OWASP Juice Shop	github.com/juice-shop/juice-shop	Target web app (via Docker)
DVWA	github.com/digininja/DVWA	Alternative vulnerable PHP app
Docker	docs.docker.com/engine/install/	Runs Juice Shop inside Kali

3. How the lab is laid out

The lab uses two network adapters on each VM. The first one is NAT, which lets the VMs access the internet for installing packages and updates. The second is a Host-Only adapter — this creates a private network (192.168.56.0/24) that only exists between my host machine and the VMs. The target never needs internet access, so its NAT adapter is mostly just there if I need to update it.

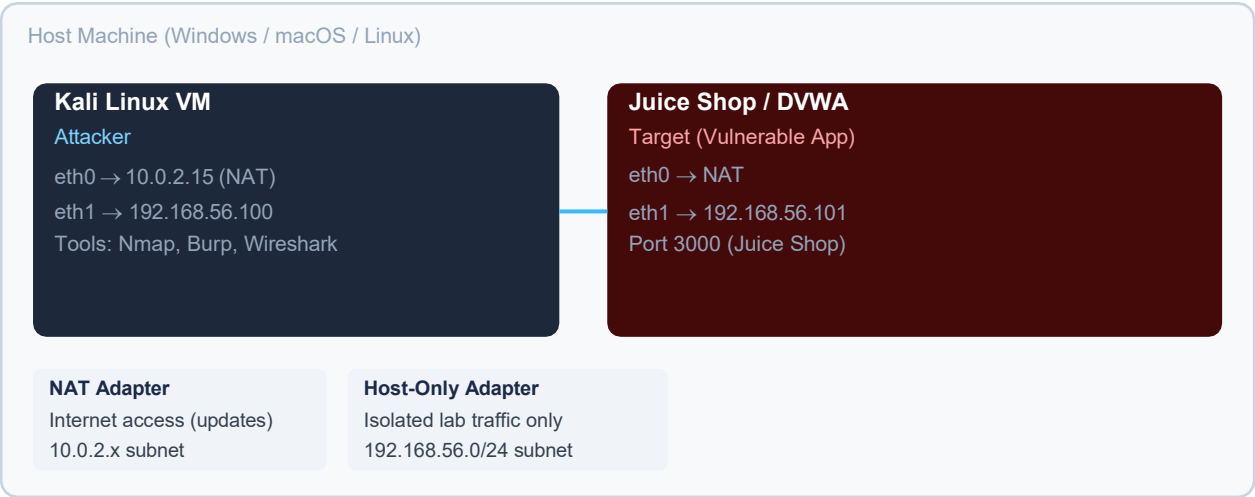


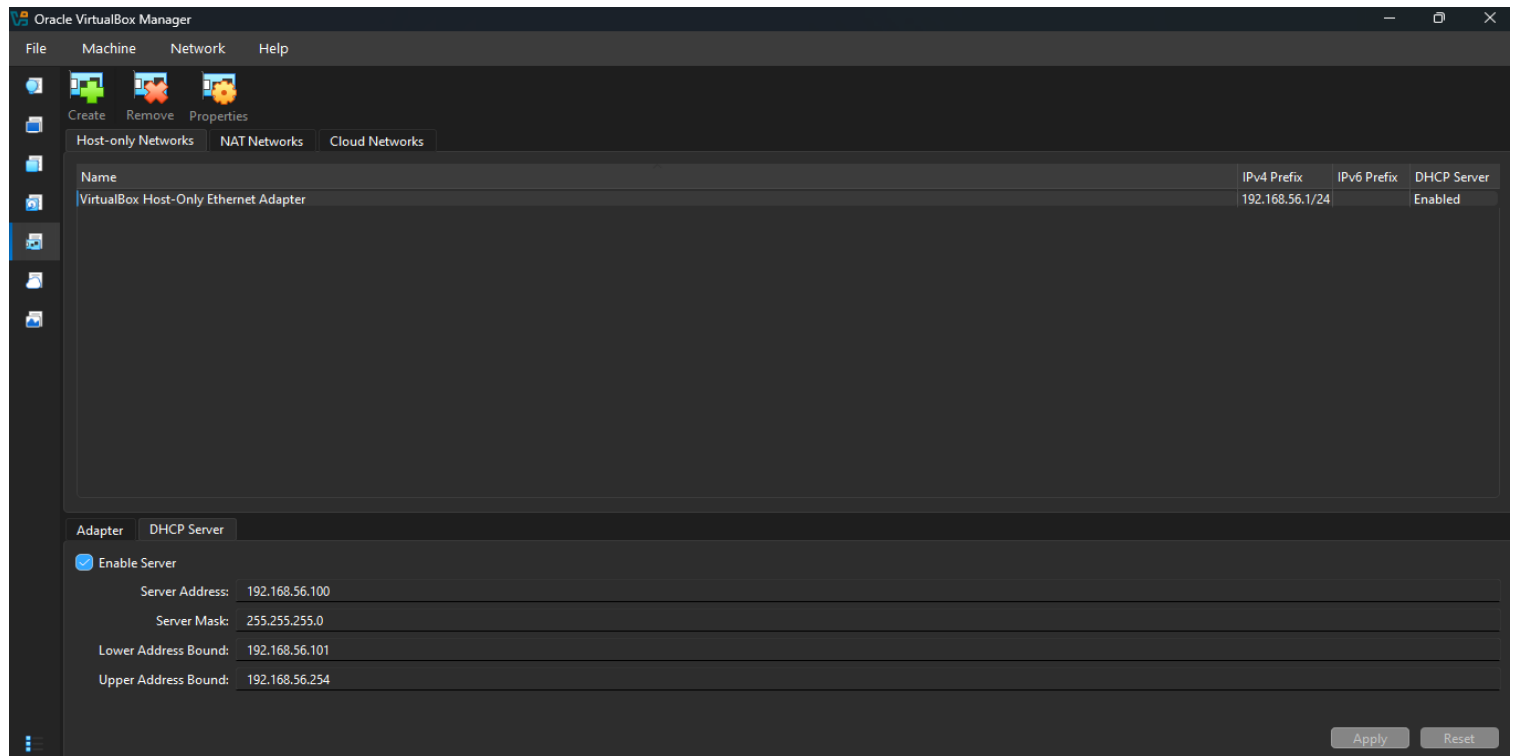
Diagram 1 — Virtual lab network layout (Host-Only subnet: 192.168.56.0/24)

Kali Linux IP	192.168.56.100 on the Host-Only adapter (eth1)
Juice Shop / DVWA IP	192.168.56.101 on the Host-Only adapter
NAT subnet	10.0.2.x — outbound internet only, VMs can't reach each other here
Host-Only subnet	192.168.56.0/24 — completely private, all attack traffic stays here

4. Step 1 — Installing VirtualBox

This was pretty straightforward. I downloaded the installer from the official site and ran it with all the default options. After that I installed the Extension Pack from the same downloads page which adds USB 2.0/3.0 support and a few other things that are useful later.

The main thing I had to do manually was create the Host-Only network. By default Virtual Box doesn't have one set up. I went to File → Host Network Manager, clicked Create, and set the address to 192.168.56.1 with subnet mask 255.255.255.0. I left the DHCP server turned on so VMs get addresses automatically.



5. Step 2 — Setting up Kali Linux

Instead of installing Kali from scratch I used the prebuilt VirtualBox OVA image from kali.org. It's the easiest way — you just import it and it's ready to go. I bumped the RAM up to 4 GB and gave it 2 CPU cores so it runs well enough for running Burp and Wireshark at the same time.

VM settings used

RAM	4096 MB
CPUs	2 cores
Storage	80 GB (dynamic)
Adapter 1	NAT — for internet/updates
Adapter 2	Host-Only Adapter → vboxnet0
Display	128 MB VRAM, 3D acceleration off

After booting up for the first time

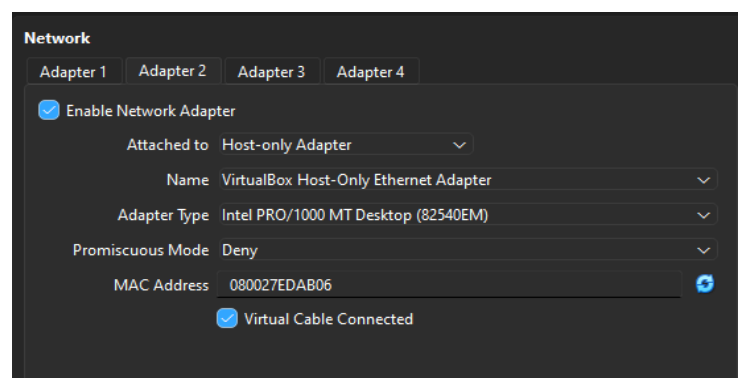
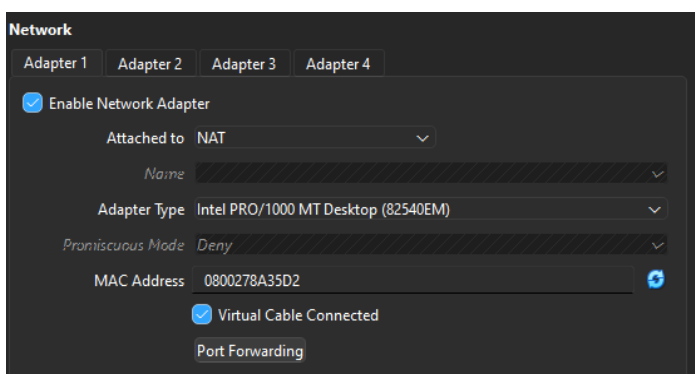
Default login is kali/kali. First thing I did was change the password, then ran the update commands below.

```
sudo apt update && sudo apt upgrade -y

# Create folder structure for the lab work
mkdir -p ~/lab/{scans,notes,pcaps,screenshots}

# Check which IPs got assigned
ip a

# Should see:
# eth0: 10.0.2.15 (NAT)
# eth1: 192.168.56.100 (Host-Only)
```



6. Step 3 — Deploying the target application

I went with OWASP Juice Shop as the main target because it covers a much wider range of vulnerabilities than DVWA and it looks like a real website rather than something obviously broken. I ran it using Docker inside the Kali VM — that way everything stays in one place and I can reset it easily by just restarting the container.

Option A (used) — Juice Shop via Docker

```
# Install Docker
sudo apt install -y docker.io
sudo systemctl enable --now docker

# Pull and start Juice Shop
sudo docker pull bkimminich/juice-shop
sudo docker run -d -p 3000:3000 --name juice bkimminich/juice-shop

# Confirm it's running
sudo docker ps

# Open it in the browser
# http://localhost:3000 (from Kali itself)
# http://192.168.56.100:3000 (if accessing from host machine)
```

Option B — DVWA (alternative, also tested)

I also set up DVWA quickly just to make sure I had a backup. It needs Apache, PHP, and MariaDB. Steps below are the short version:

```
sudo apt install -y apache2 php php-mysql mariadb-server

cd /var/www/html
sudo git clone https://github.com/digininja/DVWA.git dvwa
sudo cp dvwa/config/config.inc.php.dist dvwa/config/config.inc.php
```

```
# Then setup the DB at http://localhost/dvwa/setup.php
```

```
sudo systemctl enable --now apache2 mariadb
```

7. Step 4 — Checking connectivity

Once both machines were up I wanted to confirm the attacker could actually reach the target before moving on to any tool testing. I ran a simple ping first, then an Nmap scan.

Ping test — Kali to target

```
ping -c 4 192.168.56.101

# Output I got:
# PING 192.168.56.101: 56 data bytes
# 64 bytes from 192.168.56.101: icmp_seq=0 ttl=64 time=0.41 ms
# 64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.38 ms
# 64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.44 ms
# 64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.40 ms
# 4 packets sent, 4 received, 0% packet loss
```

Nmap host discovery on the lab subnet

```
nmap -sn 192.168.56.0/24

# Results:
# Nmap scan report for 192.168.56.1 (VirtualBox host adapter)
# Nmap scan report for 192.168.56.100 (Kali - attacker)
# Nmap scan report for 192.168.56.101 (target - Juice Shop / DVWA)
# 3 hosts up

# Quick port check on the target
```

8. Step 5 — Testing the tools

Nmap

Already used this in the connectivity check above. Also ran a version scan to see what ports were open on the target. Port 3000 came back as Node.js HTTP — which is Juice Shop, so that confirmed it.

Burp Suite

Opened Burp Suite from the Kali menu (it's preinstalled). Used the built-in Chromium browser under Proxy → Open Browser so I didn't have to mess with any Firefox proxy settings manually. Navigated to the Juice Shop login page and tried logging in — the POST request showed up straight away in the Intercept tab with the credentials in the body. Forwarded it and the login went through normally.

d

Wireshark

Started a capture on eth1 (the Host-Only interface) and browsed around Juice Shop for a minute. Could see the HTTP GET and POST requests going back and forth between my IP (192.168.56.100) and the target (192.168.56.101). Applied the filter below to clean up the view.

```
# Wireshark display filter used:
ip.addr == 192.168.56.101 && http

# Alternatively, capture from CLI:
sudo tshark -i eth1 -w ~/lab/pcaps/lab_traffic.pcapng
```

9. Evidence collected

Below is a summary of everything I collected as evidence for this task. The actual screenshots go into the evidence folder alongside this report.

#	Item	What it shows	Done
1	VM list screenshot	Both Kali and Juice Shop VMs in VirtualBox Manager	✓
2	ip a output (Kali)	eth0 at 10.0.2.15 and eth1 at 192.168.56.100 both shown	✓
3	Ping output	4/4 packets received, 0% loss to 192.168.56.101	✓
4	Nmap -sn result	3 hosts discovered on 192.168.56.0/24	✓
5	Nmap port scan	Port 3000 open, service fingerprint = Node.js HTTP	✓
6	Juice Shop in browser	http://localhost:3000 fully loaded in Kali Firefox	✓
7	Burp intercept screenshot	POST /rest/user/login captured with credentials visible	✓
8	Wireshark capture	HTTP packets between .100 and .101 on eth1	✓
9	Lab folder structure	ls -la ~/lab showing scans/notes/pcaps/screenshots	✓

10. What I learnt from this

Setting up this lab took longer than I expected, mostly because of small things like forgetting to enable the second network adapter before booting Kali the first time, and having to figure out why Docker wasn't starting (needed to enable the daemon with `systemctl`). But working through those problems actually taught me more than if everything had gone smoothly.

Main takeaways:

NAT vs Host-Only	I now actually understand why you need both. NAT is just for getting updates. The real lab traffic all flows over Host-Only where it's completely contained.
Docker is useful	Running Juice Shop as a container inside Kali means I don't need a separate VM for the target. It also resets easily — just restart the container.
Nmap basics	Even a simple <code>-sn</code> scan gives you a quick overview of what's alive. The <code>-sV</code> flag is very useful for figuring out what's actually running on open ports.
Burp proxy setup	Using the embedded browser is much easier than configuring Firefox manually, especially for a first-time setup. The Intercept tab is immediately useful.
Wireshark filters	Without a filter the capture is unreadable. Using <code>ip.addr</code> combined with the <code>http</code> protocol filter made it easy to isolate just the traffic I cared about.
Lab organisation	Having a proper folder structure from the start (scans, notes, pcaps, screenshots) saves a lot of time when I need to find something later.

Overall the lab is working exactly as it should. All five steps from the task sheet are complete and I have the screenshots to back everything up. Ready to move on to Task 3.
